

## Лабораторна робота 03

**Агент з інструментами, станом і human-in-the-loop**

Курс «Прикладні ШІ-агенти та системи на основі великих мовних моделей» · 2026/2027

|                       |                                                         |
|-----------------------|---------------------------------------------------------|
| <b>Лекції</b>         | 11, 12, 14, 15                                          |
| <b>Аудиторний час</b> | 4 год; самостійна робота 8 год                          |
| <b>Формат</b>         | індивідуально                                           |
| <b>Вартість</b>       | 0 грн: локальна модель або stub, платні API не потрібні |

**Мета**

Реалізувати агента з типізованими інструментами, явним циклом і бюджетом кроків, перетворити його на граф станів із checkpoint і додати підтвердження людини на незворотній дії.

**Теоретичні відомості**

Агентом називають систему, у якій модель сама керує потоком керування: вона вирішує, який наступний крок зробити і коли зупинитися. Це відрізняє агента від workflow, де послідовність кроків задана кодом наперед [2]. Різниця інженерна, а не маркетингова: вона каже, хто ухвалює рішення про наступний крок, і більшість продакшн-задач закривається саме workflow, бо він дешевший і передбачуваніший.

Інструмент — це контракт із шести полів: назва, призначення, схема аргументів, схема результату, побічні ефекти і права. Перші три потрапляють у запит до моделі, останні три живуть у реєстрі інструментів і перевіряються кодом до виконання. Опис інструмента є частиною промпта: модель обирає інструмент за текстом опису, тому два інструменти з майже однаковими описами дають хиткий вибір між ними, і жодне налаштування семплінгу цього не лікує.

Модель не виконує інструмент. Вона повертає прохання: структуру з іменем і аргументами. Застосунок виконує операцію і повертає результат у діалог, після чого модель продовжує. Усе, що агент може зробити, визначається обліковими даними процесу, який виконує виклик, а не намірами моделі.

ReAct додає до простору дій ще один тип кроку. Автори розширюють простір дій до  $\hat{A} = A \cup L$ , де  $L$  — простір мови: думка не змінює зовнішнього середовища і тому не дає спостереження у відповідь [1]. Практично це чергування трьох елементів: думка формулює, чого бракує; дія викликає інструмент; спостереження повертає результат у контекст. Думка коштує токенів і платить за себе тим, що наступна дія стає обґрунтованою.

Помилка інструмента є вхідними даними для моделі, а не аварією. Специфікація MCP розділяє протокольні помилки, корисні застосунку, і помилки виконання, які віддають моделі як зворотний зв'язок, придатний до дії [4]. Стек-трейс у контексті забирає крок і не дає нічого; повідомлення «поле `date`: очікується формат RRRR-MM-ДД» повертає агента на маршрут. Окремо від збою стоїть порожній результат: це успішний виклик, і поводитися з ним треба інакше.

Цикл без явного критерію зупинки — це нескінченний цикл із чужим бюджетом. Тому задаються ліміт кроків, ліміт часу і перелік дозволених дій, а вичерпання бюджету не мовчить: агент віддає зібране, позначає задачу незавершеною і передає людині.

Неявний цикл важко відновити й перевірити, тому його роблять графом: вузли — детерміновані або модельні кроки, ребра — переходи, стан — типізована структура, що передається

між вузлами. Checkpointer зберігає знімок стану після кожного супер-кроку, тому падіння процесу не відкидає маршрут на початок: відновлення продовжує з останнього знімка і повторно виконує лише вузол, перерваний посередині. Гарантія тут *at-least-once*, а не *exactly-once*, і залежить вона від режиму запису: за відкладеного запису (*durability="async"*) можна втратити й останній завершений крок, тому для відтворюваної демонстрації задають *durability="sync"*. Переривання перед вузлом запису дозволяє дочекатися людини [3]. Людина в циклі — елемент архітектури, а не кнопка в інтерфейсі: підтвердження, редагування стану і вибір гілки є трьома різними механізмами.

## Хід роботи

---

Позначка *ауд.* — крок виконується на занятті, *сам.* — самостійно в межах СРС. Аудиторна частина розрахована на 4 години.

1. **Три інструменти (50 хв, ауд.).** Калькулятор, читання з SQLite, пошук-заглушка. *Перевірка:* схеми аргументів відхиляють невалідний ввід до звертання до даних.
2. **Цикл ReAct вручну (60 хв, ауд.).** Думка, дія, спостереження — без фреймворку. *Перевірка:* у логах видно всі три елементи кожного кроку і підрахунок токенів службового обміну.
3. **Бюджети (30 хв, ауд.).** 8 викликів, 60 секунд — у коді циклу, не в промпті. *Перевірка:* у логах є щонайменше один вичерпаний бюджет із коректним завершенням, а не з винятком.
4. **Типізовані помилки (40 хв, ауд.).** Шість класів із таблиці лекції 12. *Перевірка:* з тексту помилки впливає наступна дія агента. Стек-трейс у контекст не потрапляє.
5. **Перенесення в граф (50 хв, ауд.).** Той самий цикл вузлами і ребрами. *Перевірка:* лічильник кроків живе *ключем стану*, інакше він не переживе відновлення.
6. **Checkpointing (40 хв, сам.).** Збереження в SQLite, режим запису `durability="sync"`. *Перевірка:* перервати процес сигналом усередині спеціально сповільненого *чистого* вузла і продовжити з останнього знімка. Повторно виконається лише цей вузол, а не весь маршрут.
7. **Дія із записом і людина в контурі (50 хв, сам.).** Переривання перед вузлом запису. **Найважливіша перевірка роботи:** побічний ефект стоїть *після* точки зупинки. Якщо перед — після відновлення запис створиться двічі.
8. **Ідемпотентність (30 хв, сам.).** Підтверджений запис, повторений після відновлення, не створює другого.
9. **Прогін і таблиця (50 хв, сам.).** 20 задач, з них 2 свідомо нерозв'язні. *Перевірка:* на нерозв'язних агент зупиняється за бюджетом, а не циклиться.

Крок 7 варто спробувати спершу *неправильно* — поставити запис до переривання і побачити подвійний ефект власними очима. Це найкраще запам'ятовується.

## Завдання

---

1. Реалізувати три інструменти з JSON-схемами аргументів і результату: калькулятор, читання з локальної бази SQLite лише для читання і пошук-заглушку з фіксованим набором відповідей. Для кожного заповнити всі шість полів контракту.
2. Реалізувати цикл ReAct явно, без агентного фреймворку: думка, дія, спостереження, з логуванням кожного кроку і підрахунком токенів службового обміну.
3. Задати бюджет: не більше 8 викликів інструментів і 60 секунд на задачу. Показати в логах щонайменше один випадок вичерпання бюджету і коректне завершення з частковим результатом.
4. Реалізувати типізовані помилки інструментів для шести класів: невалідний аргумент, не знайдено, немає прав, ліміт швидкості, збій сервісу, порожній результат. Показати в логах щонайменше одне успішне самовиправлення моделі після такої помилки.

5. Перенести той самий цикл у граф станів (LangGraph або власна реалізація) з не менш ніж п'ятьма вузлами і двома умовними переходами. Стан описати типізованою структурою.
6. Додати checkpointer із збереженням у SQLite і режимом `durability="sync"`. Перервати процес сигналом `kill -9` усередині чистого (без побічних ефектів) вузла і продовжити з останнього checkpoint: раніше завершені вузли не виконуються повторно, перерваний виконується наново. У звіті вказати, які кроки повторились і чому це не помилка.
7. Додати четвертий інструмент, що змінює стан (наприклад, створення запису), і поставити перед ним переривання з підтвердженням людини. Заборона має діяти на рівні графа, а не тексту промпта.
8. Перевірити ідемпотентність: підтверджений запис, повторений після відновлення з checkpoint, не має створювати другий запис. Оскільки протокол відновлення дає `at-least-once`, захистом є ключ ідемпотентності, обчислений до виклику, а не сподівання, що повтору не станеться.
9. Прогнати 20 задач, з яких не менше п'яти потребують двох і більше інструментів, а дві — свідомо нерозв'язні. Звести в таблицю: частку розв'язаних, середню кількість кроків, кількість вичерпаних бюджетів, кількість відпрацьованих типізованих помилок.
10. Здати: код і README, схему графа, лог 20 прогонів, демонстрацію відновлення після падіння, демонстрацію підтвердження людини і висновок до 200 слів про те, які кроки варто було винести з моделі в детермінований код.

## Контрольні запитання

---

1. За якою ознакою задачу віддають агенту, а не workflow?
2. Чому опис інструмента вважають частиною промпта?
3. Що саме повертає модель, коли «викликає інструмент»?
4. Чим думка в ReAct відрізняється від дії з погляду оновлення контексту?
5. Чому порожній результат виділяють окремо від збою сервісу?
6. Що має статися при вичерпанні бюджету кроків і чого статися не має?
7. Які переваги дає граф станів, яких не дає цикл `while`?
8. Після відновлення з checkpoint запис створився двічі. Де помилка?
9. Чому підтвердження людини реалізують у графі, а не інструкцією в промпті?
10. Агент повторює той самий пошук п'ять разів поспіль. Які три причини найімовірніші?

## Література

---

1. Yao S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.
2. Anthropic. Building Effective Agents. Engineering blog, 2024.
3. LangGraph. Persistence: checkpoints, threads, durability і Human-in-the-loop.
4. Model Context Protocol. Server: Tools — розділення протокольних помилок і помилок виконання.

5. Shinn N. et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS, 2023.
6. Cemri M. et al. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657, 2025.
7. OWASP GenAI Security Project. LLM06: Excessive Agency, 2025.

## **Походження шаблону**

---

Макет адаптовано з Laboratory report template for Basic Chemistry at Stockholm University, Viktor Bengtsson, LPPL 1.3c: article, Times-подібна гарнітура, широкі поля, fancyhdr, booktabs.